Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1, 3×3, 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

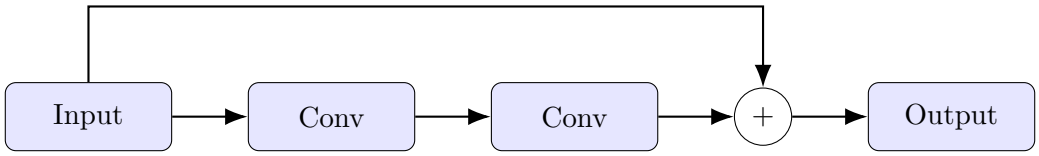

Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

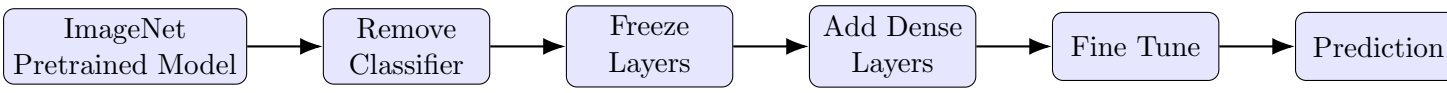

Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

GitHub Link

GitHub Repository: Deep Learning Lab - Lab 4

15. Experiment Results

Task 1: Dataset Preparation

CIFAR-10 was successfully loaded using the Hugging Face dataset repository. The dataset contains 50,000 training images and 10,000 testing images, with 10 object classes. Each image has an original size of $32 \times 32 \times 3$ and the pixel values were converted to tensors in the range $[0, 1]$.

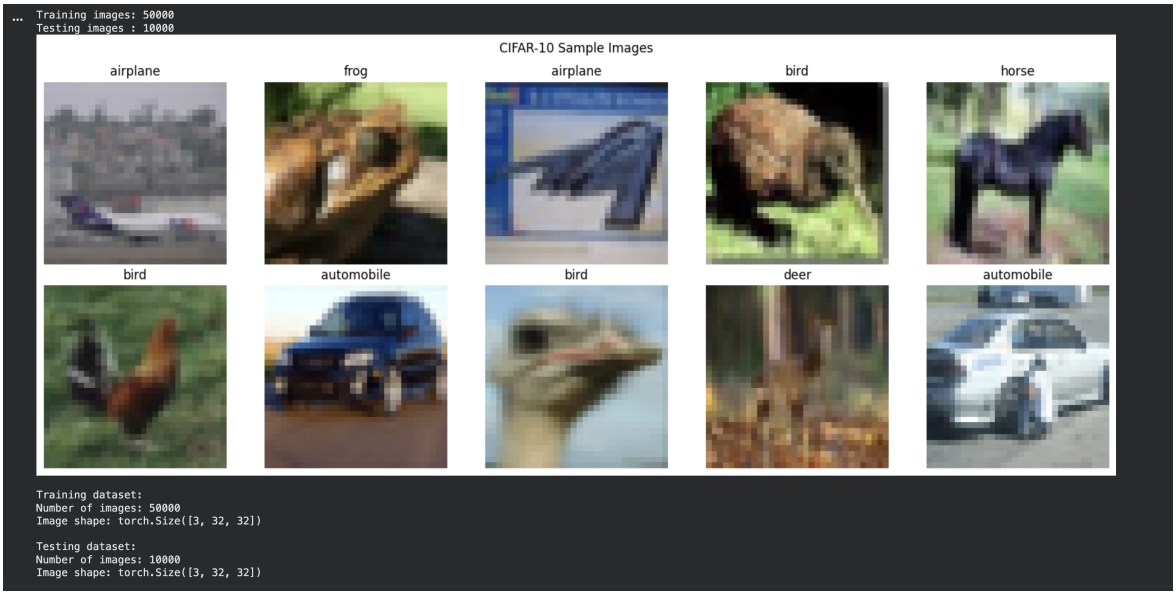

Figure 3: Sample CIFAR-10 images used for dataset preparation.

The sample images show the ten different CIFAR-10 categories and confirm that the dataset has been loaded correctly. The images are small 32×32 RGB images, which makes classification challenging because fine-grained visual details are limited.

Task 2: Transfer Learning

MobileNetV2 pretrained on ImageNet was used as the convolutional backbone. The convolutional features were initially frozen and a new classifier was added consisting of Global Average Pooling, a 128-unit ReLU layer, Dropout, and a 10-class output layer.

The model contained 2,389,130 total parameters, of which 165,258 were trainable during the initial transfer-learning stage. The model produced an output of shape 32×10, and the probability values summed to approximately 1, confirming the correctness of the Softmax output.

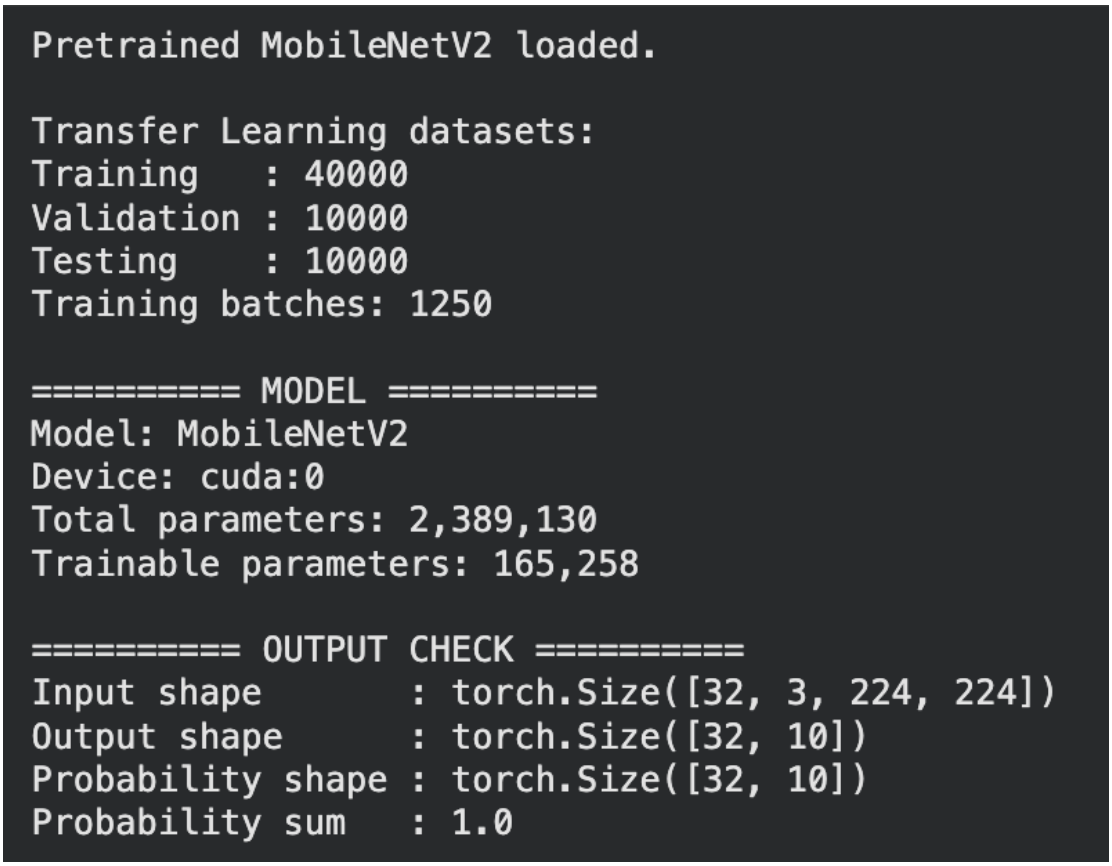

Figure 4: MobileNetV2 transfer-learning model and parameter information.

Transfer learning substantially reduces the number of parameters that must initially be trained because the pretrained convolutional features are kept frozen. The 10-dimensional output corresponds correctly to the 10 CIFAR-10 classes, confirming that the original ImageNet classifier was replaced successfully.

Task 3: Model Training

The model was trained using the Adam optimizer with a learning rate of 0.001, batch size of 32, Cross Entropy Loss, and 10 epochs.

Table 1: Task 3 training results

Metric	**Initial**	**Final**
Training Loss	1.8156	1.6263
Training Accuracy	35.99%	41.70%
Validation Loss	1.6922	1.6050
Validation Accuracy	40.15%	42.88%

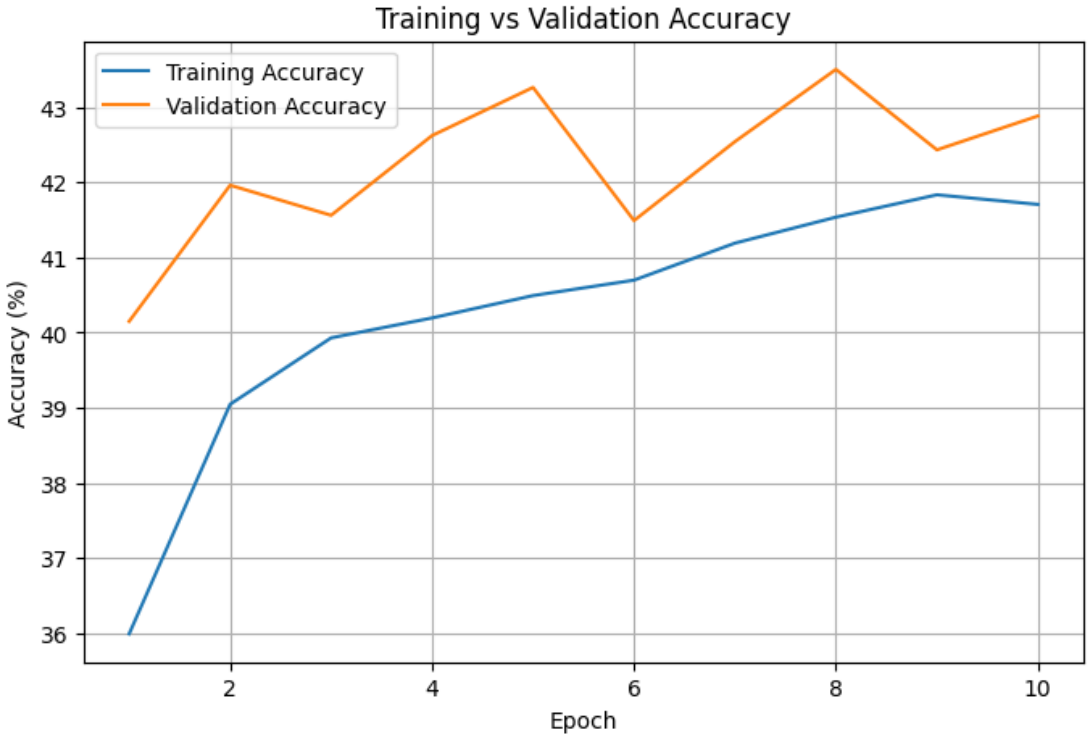

Figure 5: Training and validation accuracy during Task 3.

The training accuracy increased from 35.99% to 41.70%, while the validation accuracy increased from 40.15% to 42.88%. The decrease in both training and validation loss indicates that the classifier learned useful features, although the frozen convolutional base limited the amount of adaptation to CIFAR-10.

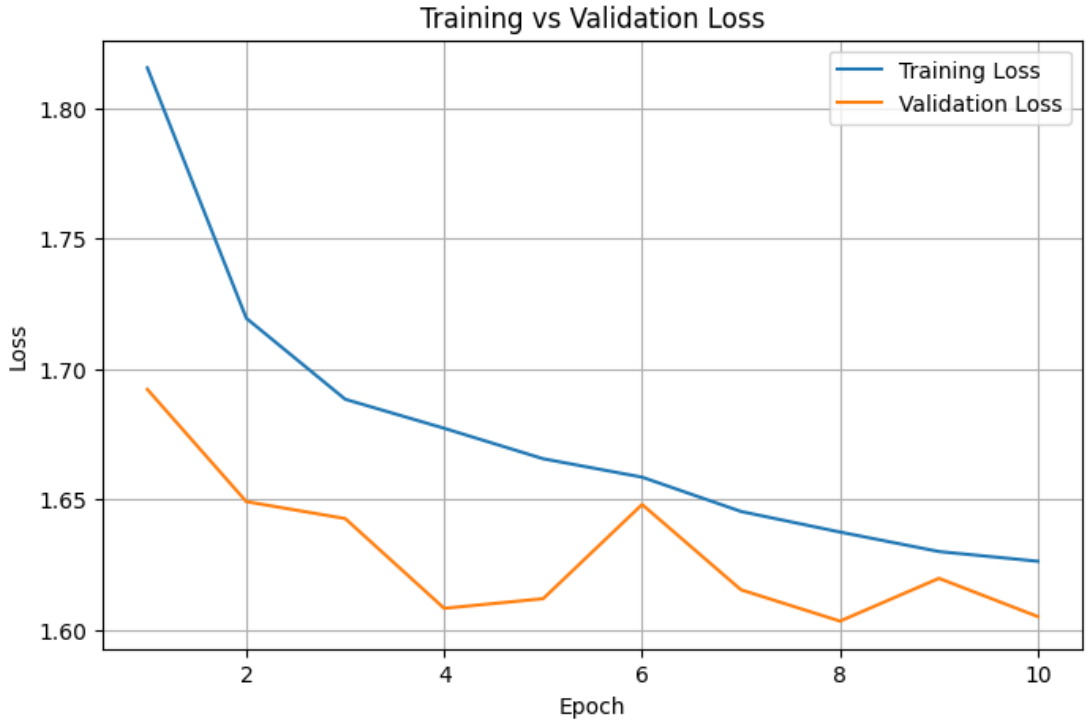

Figure 6: Training and validation loss during Task 3.

Both training and validation losses generally decreased over the training period, indicating convergence of the classifier. However, the validation accuracy remained around 40–43%, suggesting that the

ImageNet features were not sufficiently adapted to the CIFAR-10 dataset while the convolutional layers remained frozen.

Task 4: Fine Tuning

For fine tuning, the last convolutional blocks of MobileNetV2 were unfrozen. The model was then trained for another 5 epochs using a smaller learning rate of 0.0001.

Table 2: Fine-tuning results

Metric	Before Fine Tuning	After Fine Tuning
Validation Accuracy	42.88%	55.91%
Training Accuracy	41.70%	54.80%

```
Trainable parameters after unfreezing: 1691338
Epoch [1/5] | Train Loss: 1.5270 | Train Acc: 45.53% | Val Loss: 1.4283 | Val Acc: 49.64%
Epoch [2/5] | Train Loss: 1.4052 | Train Acc: 49.77% | Val Loss: 1.3719 | Val Acc: 51.86%
Epoch [3/5] | Train Loss: 1.3513 | Train Acc: 52.03% | Val Loss: 1.3167 | Val Acc: 53.37%
Epoch [4/5] | Train Loss: 1.2978 | Train Acc: 53.78% | Val Loss: 1.2876 | Val Acc: 54.29%
Epoch [5/5] | Train Loss: 1.2709 | Train Acc: 54.80% | Val Loss: 1.2653 | Val Acc: 55.91%
Before fine tuning: 42.88
After fine tuning: 55.91
```

Figure 7: Training results after fine tuning the last MobileNetV2 blocks.

Fine tuning produced a significant improvement in performance. Validation accuracy increased from 42.88% to 55.91%, an improvement of approximately 13 percentage points. This demonstrates that allowing the later convolutional layers to adapt to CIFAR-10 features is more effective than training only the newly added classifier.

Task 5: Model Evaluation

After fine tuning, the model was evaluated on the 10,000-image CIFAR-10 test set.

Table 3: Final model performance

Metric	Value
Testing Accuracy	56.00%
Precision	0.5591
Recall	0.5600
F1-score	0.5588
Training Time	5.46 minutes
Total Parameters	2,389,130

	precision	recall	f1-score	support
airplane	0.58	0.59	0.59	1000
automobile	0.62	0.59	0.61	1000
bird	0.48	0.43	0.46	1000
cat	0.45	0.44	0.44	1000
deer	0.53	0.54	0.53	1000
dog	0.56	0.54	0.55	1000
frog	0.57	0.68	0.62	1000
horse	0.60	0.58	0.59	1000
ship	0.60	0.62	0.61	1000
truck	0.59	0.58	0.59	1000
accuracy			0.56	10000
macro avg	0.56	0.56	0.56	10000
weighted avg	0.56	0.56	0.56	10000

Figure 8: Confusion matrix of the fine-tuned MobileNetV2 model.

The final model achieved a testing accuracy of 56.00%, with precision, recall, and F1-score all close to 0.56. This indicates moderate classification performance across the ten classes. The model performs relatively well on classes such as frog, automobile, and ship, while bird and cat are more difficult to classify, indicating greater confusion between visually similar classes.

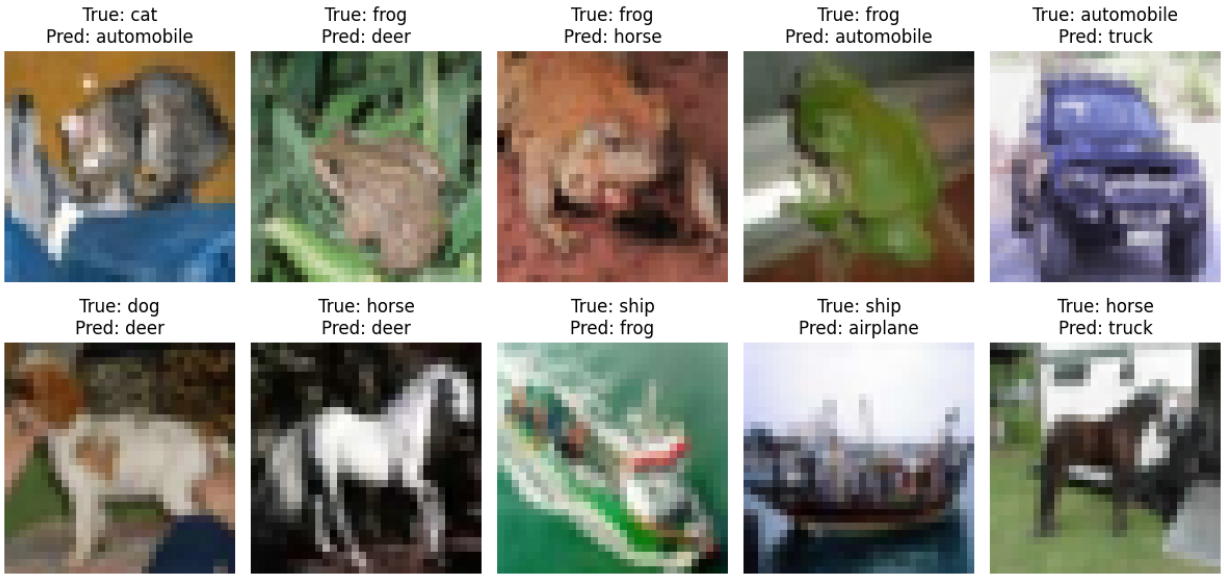

Figure 9: Examples of misclassified CIFAR-10 test images.

The misclassified examples show that the model mainly struggles with visually similar categories. The low resolution of CIFAR-10 images makes distinguishing classes such as cats, dogs, birds, and other animals more difficult.

16. Hyperparameter Study

Inference

The hyperparameter study shows that a learning rate of 0.001 and batch size of 32 gave better performance than the other tested settings. Adam performed significantly better than SGD, achieving 70.0% compared to 9.2%. Increasing the dense units to 256 did not improve accuracy. The best result was obtained with **partial freezing**, achieving a validation accuracy of **78.2%**.

HYPERPARAMETER STUDY RESULTS

	Learning Rate	Batch Size	Epochs	Optimizer	Dense Units	Frozen Layers	Validation Accuracy (%)	Time (min)
0	0.0010	32	3	Adam	128	All	70.0	0.343688
1	0.0001	32	3	Adam	128	All	61.2	0.343661
2	0.0010	16	3	Adam	128	All	68.6	0.331447
3	0.0010	64	3	Adam	128	All	69.8	0.377357
4	0.0010	32	5	Adam	128	All	68.0	0.565422
5	0.0010	32	3	SGD	128	All	9.2	0.344210
6	0.0010	32	3	Adam	256	All	69.4	0.335260
7	0.0010	32	3	Adam	128	Partial	78.2	0.366236

Figure 10: Hyperparameter study results

17. Mandatory Plots

Include the following plots in the laboratory report.

1. Sample CIFAR-10 images
2. Training Accuracy
3. Validation Accuracy
4. Training Loss
5. Validation Loss
6. Confusion Matrix
7. Misclassified Images (Optional)

Students should write a brief inference (2–3 lines) for each figure.

18. Results

0.1 Performance Metrics

Metric	**Value**
Training Accuracy	56.00%
Testing Accuracy	56.00%
Precision	0.5591
Recall	0.5600
F1-score	0.5588
Training Time	5.46 minutes
Total Parameters	2,389,130

Table 4: Final model performance

18.2 Comparison of CNN Architectures

```
================================================================
CNN ARCHITECTURE COMPARISON
================================================================
```

	Model	Parameters	Accuracy (%)	Training Time (min)
0	LeNet-5	62006	24.2	0.102468
1	AlexNet	57044810	65.6	0.125069
2	VGG16	134301514	72.5	0.420473
3	GoogleNet	9960638	62.6	0.165345
4	ResNet50	23528522	61.8	0.264938

Figure 11: Comparison of CNN architectures

The results show that VGG16 achieved the highest accuracy of 72.5%, while LeNet-5 had the lowest accuracy of 24.2%. VGG16 also has the largest number of parameters, whereas LeNet-5 is the most lightweight and fastest model. Overall, the results show a trade-off between model complexity, training time, and classification accuracy.

19. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**
   AlexNet substantially improved image classification performance and popularized the use of ReLU activation, dropout, and GPU-based training.
2. **Why does VGG16 use only 3×3 convolution filters?**
   VGG16 uses stacked 3×3 filters to build deeper feature representations efficiently while keeping the convolutional operations relatively simple.
3. **Explain the advantages of the Inception module.**
   The Inception module performs parallel operations using multiple kernel sizes, allowing the network to capture features at different scales.
4. **What is the purpose of residual learning?**
   Residual learning uses skip connections to improve gradient flow and make it easier to train very deep neural networks.
5. **Differentiate LeNet and ResNet.**
   LeNet is a small and early CNN architecture designed for simple image recognition tasks, whereas ResNet is a much deeper architecture that uses residual connections.
6. **What is Transfer Learning?**
   Transfer learning reuses a model pretrained on a large dataset, such as ImageNet, and applies its learned features to a new target dataset.
7. **Why is fine tuning required?**
   Fine tuning adapts selected pretrained features to the target dataset, improving the model's ability to perform the specific task.
8. **Explain the difference between dilated convolution and transpose convolution.**
   Dilated convolution expands the receptive field without significantly increasing the number of parameters, while transpose convolution performs learnable upsampling.
9. **Why do pretrained models converge faster?**

Pretrained models converge faster because useful visual features have already been learned from a large dataset.

10. **Compare the computational complexity of LeNet and ResNet.**
    LeNet is much smaller and computationally cheaper than ResNet, while ResNet is significantly deeper and requires more computational resources.

20. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: `https://www.tensorflow.org`
8. Keras Documentation: `https://keras.io`